

Manual de instalacion

1. Objetivo

Este manual describe los pasos necesarios para instalar, ejecutar y desplegar el proyecto Avatares DevOps en un entorno local y en AWS EKS. Su objetivo es que cualquier persona pueda replicar el entorno sin depender de conocimiento previo del proyecto.

2. Descripcion breve del proyecto

El proyecto parte de un generador de avatares open source y fue adaptado con practicas DevOps para cubrir:

- frontend React servido con Nginx
- backend Flask con Gunicorn
- persistencia en SQLite
- contenedores Docker
- despliegue en Kubernetes
- infraestructura como codigo con Terraform
- observabilidad con Prometheus y Grafana
- pipeline CI/CD con GitHub Actions

3. Arquitectura general

La arquitectura funcional se compone de:

- navegador del usuario
- servicio web basado en Nginx que sirve la SPA y hace proxy al backend
- API Flask/Gunicorn para generacion y galeria de avatares
- base SQLite para persistencia
- clúster EKS para despliegue en staging/production
- Prometheus para scrape de metricas
- Grafana para visualizacion
- Terraform para backend remoto, cluster y entornos

Flujo principal:

1. El usuario accede al frontend.
2. El frontend consume la API por `/api`.
3. La API genera avatares y guarda registros en SQLite.
4. Prometheus scrapea `/metrics`.
5. Grafana consume Prometheus como datasource.

4. Prerrequisitos

4.1 Para entorno local

Instalar previamente:

- Git
- Docker y Docker Compose
- Make
- curl
- Python 3.12 o compatible
- Node.js 22 o compatible

4.2 Para infraestructura y clúster

Instalar ademas:

- Terraform
- AWS CLI v2
- kubectl

4.3 Para CI/CD

Tener acceso al repositorio GitHub y permisos para configurar:

- GitHub Actions
- GitHub Container Registry (GHCR)
- GitHub Secrets y Variables

5. Clonacion del repositorio

```
git clone https://github.com/afedele-devops/avatares-devops.git
cd avatares-devops
```

6. Instalacion y ejecucion local con Docker

Esta es la forma mas rapida de replicar el entorno de desarrollo.

6.1 Construir y levantar servicios

```
make up
```

6.2 Verificar estado

```
make health
make test-api
```

6.3 Acceso local

Abrir en navegador:

- <http://localhost:8080>

6.4 Detener servicios

```
make down
```

7. Ejecucion local sin Docker

7.1 Backend

```
cd api
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python install_parts.py
export FLASK_APP=app.py
export DB_PATH=./avatars.db
flask run
```

Backend disponible en:

- <http://localhost:5000>

7.2 Frontend

En otra terminal:

```
cd web
npm install
npm run dev
```

Frontend disponible en:

- <http://localhost:5173>

8. Ejecucion de pruebas

8.1 Pruebas del backend

```
make test-backend
```

8.2 Pruebas del frontend

```
make test-frontend
```

8.3 Todas las pruebas

```
make test
```

9. Provisionamiento de infraestructura con Terraform

9.1 Estructura de infraestructura

- `infra/bootstrap`: crea backend remoto de Terraform
- `infra/modules/eks`: modulo reusable del cluster
- `infra/envs/staging`: entorno de staging
- `infra/envs/production`: entorno de produccion

9.2 Configurar AWS CLI

Antes de usar Terraform o kubectl con EKS, configurar AWS CLI:

```
aws configure
aws sts get-caller-identity
```

Se recomienda usar region:

- `us-east-1`

9.3 Crear backend remoto

```
cd infra/bootstrap
terraform init
terraform plan
terraform apply
```

Este stack crea:

- bucket S3 para state
- tabla DynamoDB para locking

9.4 Inicializar entorno staging

```
cd ../envs/staging
terraform init -reconfigure \
  -backend-config="bucket=<STATE_BUCKET>" \
  -backend-config="key=avatares-devops/staging/terraform.tfstate" \
  -backend-config="region=<AWS_REGION>" \
  -backend-config="dynamodb_table=<LOCKS_TABLE>" \
  -backend-config="encrypt=true"
terraform plan
terraform apply
```

9.5 Inicializar entorno production

```
cd ../production
terraform init -reconfigure \
  -backend-config="bucket=<STATE_BUCKET>" \
  -backend-config="key=avatares-devops/production/terraform.tfstate" \
  -backend-config="region=<AWS_REGION>" \
  -backend-config="dynamodb_table=<LOCKS_TABLE>" \
  -backend-config="encrypt=true"
terraform plan
terraform apply
```

10. Conexion al cluster EKS

Despues del terraform apply del entorno:

```
aws eks update-kubeconfig --region us-east-1 --name avatares-staging
kubectl get nodes
```

11. Despliegue de la aplicacion en Kubernetes

11.1 Crear namespace si no existe

```
kubectl get ns avatares-staging >/dev/null 2>&1 || kubectl create ns avatares-staging
```

11.2 Crear imagePullSecret para GHCR

Si las imagenes son privadas en GHCR, crear el secret manualmente:

```
kubectl -n avatares-staging create secret docker-registry ghcr-secret \
  --docker-server=ghcr.io \
  --docker-username=<GITHUB_USER> \
  --docker-password=<GITHUB_TOKEN_CON_READ_PACKAGES> \
  --docker-email=<EMAIL>
```

11.3 Aplicar manifests

```
kubectl -n avatares-staging apply -k k8s/
```

11.4 Validar despliegue

```
kubectl -n avatares-staging get pods -o wide
kubectl -n avatares-staging get svc -o wide
kubectl -n avatares-staging get ingress -o wide
kubectl -n avatares-staging get pvc -o wide
```

12. Acceso a la aplicacion desplegada

Obtener el hostname del ingress:

```
kubectl -n avatares-staging get ingress avatares-ingress -o wide
```

Abrir en el navegador la URL del ADDRESS usando http.

Verificaciones recomendadas:

- `http://<HOST>/health`
- `http://<HOST>/api/avatar/spec`

13. Observabilidad

13.1 Componentes desplegados

El overlay `k8s/observability` incluye:

- Prometheus
- Grafana
- datasource de Prometheus
- dashboard `avatars` desde ConfigMap

13.2 Validar scraping de metricas

La API expone annotations compatibles con Prometheus:

- `prometheus.io/scrape=true`
- `prometheus.io/path=/metrics`
- `prometheus.io/port=5000`

13.3 Acceder a Grafana

```
kubectl -n avatares-staging port-forward svc/grafana 3000:3000
```

Abrir:

- `http://localhost:3000`

Credenciales configuradas en manifests:

- usuario: `admin`
- password: `admin`

Ruta sugerida:

- Dashboards
- Browse
- carpeta `Avatares`
- dashboard `avatars`

14. Pipeline CI/CD

El pipeline esta definido en:

- `.github/workflows/ci-cd.yml`

Secuencia principal:

1. pruebas backend
2. pruebas frontend
3. build de imagenes

4. publish a GHCR
5. deploy a staging
6. smoke test post deploy

14.1 Configuración requerida en GitHub

Secrets requeridos:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- KUBE_CONFIG

Variables requeridas o recomendadas:

- AWS_REGION
- STAGING_NAMESPACE
- STAGING_URL

14.2 Regla actual del workflow

La publicación de imágenes, el deploy y el smoke test están configurados para ejecutarse solo en push a main.

15. Evidencias generadas

Archivos útiles ya incluidos en el repositorio:

- docs/evidencias/iac/terraform-fmt.txt
- docs/evidencias/iac/terraform-validate-staging.txt
- docs/evidencias/iac/terraform-validate-production.txt
- docs/evidencias/k8s/get-pods-staging.txt
- docs/evidencias/k8s/get-svc-staging.txt
- docs/evidencias/k8s/get-ingress-staging.txt
- docs/evidencias/k8s/get-pvc-staging.txt
- docs/evidencias/k8s/staging-url-candidate.txt

16. Troubleshooting rápido

16.1 AWS CLI no valida credenciales

```
aws sts get-caller-identity
```

Si falla, volver a ejecutar:

```
aws configure
```

16.2 kubectl devuelve Unauthorized

Recargar kubeconfig:

```
aws eks update-kubeconfig --region us-east-1 --name avatares-staging
```

16.3 Pods en ImagePullBackOff

Verificar:

- existencia de ghcr-secret
- visibilidad o permisos de las imágenes GHCR
- nombre de imagen y tag publicados

16.4 PVC no enlaza

Verificar:

- existencia de StorageClass
- addon EBS CSI activo
- permisos IAM del driver

17. Validacion del requerimiento de documentacion tecnica

Requerimiento evaluado:

- manual de instalacion
- guia de despliegue
- explicacion de la arquitectura
- claridad suficiente para replicar el entorno

Resultado:

- Antes de este archivo: cumplimiento parcial, porque la informacion existia pero estaba fragmentada entre `README.md`, `infra/README.md`, `infra/bootstrap/README.md` y documentos de evidencia.
- Con este archivo: el requerimiento queda cubierto de forma sustancial, porque ahora existe una guia paso a paso, centralizada y replicable.

Observacion final:

- El requerimiento puede considerarse cumplido, siempre que junto con este manual se entreguen las evidencias visuales y de ejecucion ya generadas en `docs/evidencias/`.